Relatório Técnico Final – Etapa 1 Revisada & Etapa 2 Completa (N2)

Universidade Católica de Brasília (UCB)

- **Curso**: Bacharelado em Engenharia de Software
- **Unidade Curricular**: Laboratório de Banco de Dados
- **Docente**: Prof. Samuel Novais Moura Júnior
- **Ano/Semestre**: 2026/2 — Turma `GPE17M40083`
- **Tema do Projeto**: Sistema de Gestão de Contas e Operações Bancárias
- **SGBD Utilizado**: MySQL 8.0 (Conteinerizado em Docker)

Integrantes da Equipe

- **Gabriel Almeida Silva Netto**
- **Gabriel Reis**
- **Gabriel Lima Leite**
- **Celiandro Borges Mazarro**
- **Caio Gasparin**
- **Eduardo Maciel**

1. Tabela de Rastreabilidade das Regras de Negócio (Devolutiva C5)

Regra	Enunciado da Regra de Negócio	Mecanismo de Implementação no Banco / Aplicação
RN01	Todo cliente deve possuir identificação fiscal única (CPF ou CNPJ válido).	Restrição `uq_cliente_cpf_cnpj` `UNIQUE` e `NOT NULL` na tabela `cliente`.
RN02	Toda conta bancária deve estar vinculada a uma agência física cadastrada.	Chave estrangeira `fk_conta_agencia` com `ON DELETE RESTRICT`.
RN03	Cada setor da agência pertence a uma agência e possui nome descritivo.	Chave estrangeira `fk_setor_agencia` e constraint `pk_setor`.
RN04	Funcionários possuem cargos predefinidos e estrutura hierárquica de supervisão.	Autorrelacionamento `fk_func_supervisor` e `ck_func_cargo`.
RN05	O histórico de atuação de funcionários em setores registra vigência temporal.	Tabela associativa `funcionario_setor` com PK composta e atributos próprios.
RN06	Contas bancárias operam nas modalidades Corrente ou Poupança e possuem controle de status.	Constraints `ck_conta_modalidade` e `ck_conta_status`.
RN07	Uma conta bancária pode ter múltiplos titulares, classificados por tipo de vínculo.	Tabela associativa `cliente_conta` com constraint `ck_cc_tipo_titular`.
RN08	Toda transição de status de conta gera registro histórico temporal inalterável.	Entidade fraca `historico_status_conta` e trigger `trg_audit_conta_bancaria`.
RN09	Transações financeiras devem ter valor positivo e dados de origem/destino válidos.	Constraint `ck_trans_valor` `CHECK (valor > 0)` e `ck_trans_tipo`.

Regra	Enunciado da Regra de Negócio	Mecanismo de Implementação no Banco / Aplicação
RN10	Contratos de empréstimo devem estar vinculados a clientes com status controlado.	Chave estrangeira `fk_emp_cliente` e constraint `ck_emp_status`.
RN11	Empréstimos consignados exigem indicação do órgão público emissor e margem.	Tabela por subclasse `emprestimo_consignado` com campos obrigatórios.
RN12	Financiamentos exigem especificação do bem financiado e valor de entrada.	Tabela por subclasse `financiamento` com validação de entrada.
RN13	Empréstimos pessoais registram finalidade e taxa de juros pré-fixada.	Tabela por subclasse `emprestimo_pessoal` com taxa pré-definida.
RN14	Contratos de empréstimo desdobram-se em parcelas datadas com controle de quitação.	Entidade fraca `parcela_emprestimo` com chave composta e trigger de auditoria.
RN15	Cartões são emitidos para contas correntes com bandeira e tipo regulamentados.	Tabela `cartao` com constraint `ck_cartao_tipo` e `ck_cartao_status`.
RN16	Correntistas podem cadastrar favorecidos bancários frequentes.	Tabela `beneficiario` com chave estrangeira apontando para a conta.
RN17	Transferências entre contas devem debitar a origem e creditar o destino atomicamente.	Transação explícita com Prepared Statements e `COMMIT` / `ROLLBACK`.
RN18	Tentativas de injeção de SQL devem ser bloqueadas nativamente.	Uso obrigatório de parâmetros em prepared statements na camada `app/db.py`.
RN19	Acesso operacional deve respeitar o princípio do menor privilégio por perfis.	Usuários MySQL nativos `user_admin`, `user_atendente` e `user_auditor`.
RN20	Dados de contato e CPFs de correntistas devem ser protegidos contra exposição.	Visão `vw_cliente_atendimento` com mascaramento de CPF e diretrizes LGPD.
RN21	Cada empréstimo deve possuir status definido.	Campo obrigatório `emprestimo.status` com `CHECK` para os estados permitidos: em análise, ativo, quitado, inadimplente e cancelado.
RN22	Um cliente pode possuir no máximo N empréstimos ativos simultaneamente; o limite deve ser validado antes da concessão.	Validar a quantidade de empréstimos com status ativo do cliente antes de inserir/ativar novo contrato. Implementar na aplicação ou em procedimento/trigger transacional para evitar ultrapassar o limite em operações concorrentes; N deve ser parametrizado conforme a regra adotada.
RN23	Um cliente pode possuir um ou mais empréstimos, e cada contrato pertence a um único cliente.	FK obrigatória `emprestimo.id_cliente` para `cliente`; cada linha de empréstimo referencia um único cliente.
RN24	Cada empréstimo deve possuir uma ou mais parcelas dependentes.	Tabela `parcela_emprestimo` com FK `id_emprestimo` e chave composta (`id_emprestimo`, `numero_parcela`). A aplicação deve garantir a criação de ao menos uma parcela ao concluir o contrato.

Regra	Enunciado da Regra de Negócio	Mecanismo de Implementação no Banco / Aplicação
RN25	Cada parcela deve possuir número, valor, vencimento, eventual data de pagamento e status.	Campos de `parcela_emprestimo` com chave composta, `valor`, `data_vencimento`, `data_pagamento` opcional e `status` restringido por `CHECK`; trigger de auditoria pode registrar alterações de status/pagamento.
RN26	Uma conta pode possuir um ou mais cartões, e cada cartão está associado a uma única conta.	Tabela `cartao` com FK obrigatória `numero_conta` para `conta_bancaria`, permitindo vários cartões por conta.
RN27	Todo cartão deve possuir tipo: débito, crédito ou múltiplo.	Campo obrigatório `cartao.tipo` com restrição `CHECK` para os valores `'Débito'`, `'Crédito'` e `'Múltiplo'`.
RN28	Todo cartão deve possuir status.	Campo obrigatório `cartao.status` com restrição `CHECK` para os estados definidos, como ativo, bloqueado, cancelado e aguardando desbloqueio.
RN29	Uma conta pode cadastrar beneficiários autorizados para facilitar transferências.	Tabela `beneficiario` com FK obrigatória `numero_conta_origem` para `conta_bancaria`, identificador próprio e campos para nome, documento e dados bancários do beneficiário.

2. Implementação da Etapa 2 (Artefatos B1 a B8)

2.1 B1: Aplicação Funcional com Interface

- Desenvolvida em **Python + Streamlit**, organizada em 7 módulos navegáveis.
- **4 CRUDs Completos**: Cliente, Conta Bancária, Cartão e Beneficiário.
- **Tratamento Amigável de Erros**: O módulo `app/db.py` intercepta códigos MySQL (`1062`, `1452`, `3819`) e exibe mensagens compreensíveis (ex: alerta de CPF duplicado, agência inexistente ou saldo insuficiente).
- **2 Telas de Consulta Parametrizada**: Extrato por período com totalizadores de volume e painel de carteira de crédito.
- **Relatório via VIEW**: Consumo direto da visão `vw_relatorio_consolidado_agencias`.

2.2 B2: Camada de Acesso a Dados

- Todas as consultas executadas com tuplas de parâmetros (`cursor.execute(sql, params)`). Concatenação de strings rigorosamente banida.
- **Resistência a SQL Injection**: Teste realizado na interface com a string `' OR '1'='1`. O prepared statement tratou a entrada como texto literal, não ocorrendo vazamento de registros.
- **Credenciais Seguras**: Gerenciadas via `.env` carregado por `python-dotenv`, com o arquivo real bloqueado no `.gitignore`.

2.3 B3: Transações e Controle de Concorrência

- **Operação Multitabela Central**: Transferência bancária entre contas:
    1. `SELECT ... FOR UPDATE` para bloqueio de linha;
    2. Débito na conta de origem;
    3. Crédito na conta de destino;
    4. Inserção na tabela `transacao`;
    5. `COMMIT` ou `ROLLBACK`.

- **Cenário de Falha Forçada**: Botão na interface simulando falha proposital no meio da transação. Comprovou-se que o banco efetuou `ROLLBACK` completo e nenhum centavo foi desviado ou perdido.
- **Experimento de Concorrência (`scripts/experimento_concorrencia.py`)**:
  - *Modo Padrão (Bloqueio em Linha)*: Duas threads simultâneas disputando a mesma conta bancária sob nível `REPEATABLE READ`. A Sessão 1 adquiriu lock de linha (`FOR UPDATE`) por 3 segundos. A Sessão 2 entrou em espera automática por 2.51 segundos e foi desbloqueada imediatamente após o `COMMIT` da primeira, lendo o saldo atualizado com zero *Lost Update*.
  - *Modo Deadlock Didático (`--deadlock`)*: Duas sessões bloqueando contas cruzadas em ordem invertida. O MySQL 8.0 detectou o ciclo de espera mútua imediatamente, disparando o erro `1213 (ER_LOCK_DEADLOCK)` e elegendo automaticamente uma das sessões como vítima, efetuando o `ROLLBACK` da transação da vítima antes de qualquer comando explícito.

2.4 B4: Otimização de Consultas (EXPLAIN ANALYZE)

Estudo de 3 consultas críticas com massa sintética comprovada de 10.123 transações (requisito: ≥ 10.000 registros):

Tabela	Quantidade Real de Registros	Finalidade no Benchmark
`transacao`	10.123	Carga massiva para teste de `filesort` e Index Range Scan
`parcela_emprestimo`	84	Recuperação de crédito e inadimplência
`historico_status_conta`	40	Trilha cronológica de auditoria de contas

1. **Consulta 1 (Extrato por Período)**:
   - *Antes*: Full Table Scan em 10.123 linhas com etapa onerosa de `filesort` (tempo ~18.4 ms).
   - *Depois (Índice `idx_transacao_conta_data`)*: Index Range Scan em ordem reversa, examinando apenas 226 linhas com tempo de **1.51 ms** (redução de 92% no tempo de resposta).
2. **Consulta 2 (Cobrança de Parcelas em Atraso — `idx_parcela_status_vencimento`)**:
   - Acesso direto às parcelas inadimplentes com predicado composto (`status = 'EM_ATRASO' AND data_vencimento < ...`), eliminando varredura sequencial.
3. **Consulta 3 (Auditoria de Status — `idx_historico_conta_data`)**:
   - Leitura indexada cronológica de alterações por conta bancária.

2.5 B5: Segurança e Controle de Acesso

- **3 Perfis de Usuário no MySQL**:
  - `user_admin`: `ALL PRIVILEGES` para manutenção e parametrização.
  - `user_atendente`: `SELECT`, `INSERT`, `UPDATE` nas tabelas operacionais e na visão mascarada; comandos de exclusão (`DELETE`) desabilitados pelo SGBD.
  - `user_auditor`: `SELECT` exclusivo em `tb_auditoria_sistema`, histórico e transações; qualquer tentativa de alteração é rejeitada pelo SGBD.
- **Senhas com Hash e Sal (bcrypt)**:
  - Implementadas com o algoritmo `bcrypt` (12 rounds de sal aleatório) no módulo `app/auth.py`.
  - Interface visual dedicada no Menu 5 ("Senhas com Hash e Sal"), permitindo gerar hashes de 60 caracteres (`$2b$12$...`) e testar autenticações válidas e inválidas em tempo real.
- **Conformidade LGPD**:
  - Dados pessoais tratados: CPF/CNPJ, nome, telefone e e-mail.

- Base Legal: Execução de contrato bancário (Art. 7º, V da LGPD) e cumprimento de obrigação legal/regulatória perante o Banco Central (Art. 7º, II).
- Medida de segurança: Acesso operacional realizado através da view `vw_cliente_atendimento`, que mascara os dígitos centrais do CPF (`***.456.789-**`).

2.6 B6: Gatilhos de Auditoria

- Tabela `tb_auditoria_sistema` registrando autor, `data_hora`, operação, tabela, chave e valores anteriores/posteriores em JSON.
- **Trigger 1 (`trg_audit_conta_bancaria`)**: Disparado `AFTER UPDATE` sempre que o status, o saldo ou a agência de uma conta sofrem mutação.
- **Trigger 2 (`trg_audit_parcela_emprestimo`)**: Disparado `AFTER UPDATE` sempre que uma parcela tem seu status alterado ou pagamento registrado.

2.7 Validação Automatizada de Ponta a Ponta (scripts/smoke_test_etapa2.py)

Para assegurar 100% de confiabilidade na demonstração ao vivo, foi desenvolvido um script de smoke test contendo 10 verificações rigorosas:

1. Conexão ativa com o banco `sistema_bancario`.
2. Massa de dados validada (> 10.000 transações).
3. Prepared Statements e bloqueio de SQL Injection.
4. Operação dos 4 CRUDs (`SELECT`s básicos).
5. Execução das 2 consultas parametrizadas.
6. Consulta à VIEW de relatório consolidado.
7. Transação multitabela com teste real de `COMMIT` e `ROLLBACK` forçado.
8. Três logins reais nos perfis MySQL comprovando privilégios concedidos e rejeitados.
9. Disparo de Trigger e validação do payload JSON em `tb_auditoria_sistema`.
10. Geração e validação do algoritmo `bcrypt` (60 caracteres com sal).

Resultado da suíte: **10/10 verificações aprovadas com louvor.**

3. Projeto Físico e Armazenamento (B7)

1. **Revisão dos Tipos de Dados**:
   - Valores monetários padronizados em `DECIMAL(15, 2)` (precisão exata sem erros de arredondamento de ponto flutuante).
   - Campos de status restritos por constraints `CHECK`, dispensando tabelas lookup adicionais desnecessárias para enums estáveis.
   - Chaves textuais regulamentadas em `VARCHAR(20)` e inteiros em `INT` (4 bytes, suportando até 2 bilhões de linhas).
2. **Estimativa de Crescimento em 2 Anos**:
   - Base projetada: 50.000 correntistas, gerando média de 4 transações diárias.
   - Volume anual de transações: `50.000 * 4 * 365 = 73.000.000` registros/ano.
   - Tamanho por registro da tabela `transacao`: ~90 bytes.
   - Projeção de armazenamento: ~6,5 GB/ano para transações e ~1,2 GB/ano para índices, totalizando ~15,4 GB em 2 anos de operação.
3. **Plano de Backup e Recuperação (Estratégia 3-2-1)**:
   - **Backup Completo**: Executado semanalmente aos domingos via snapshot físico / `mysqldump` com retenção de 4 semanas.
   - **Backup Diferencial/Incremental**: Diário às 02h00.

- **Logs Binários (Binlogs)**: Ativos continuamente para recuperação ponto-a-ponto (*Point-in-Time Recovery - PITR*) com RPO < 5 minutos.
- **Teste Periódico de Restore**: Rotina automatizada mensal que sobe o backup em container isolado e valida a integridade referencial.

4. **Proposta de Particionamento**:
   - Particionamento por faixa anual na tabela `transacao`:

```
ALTER TABLE transacao PARTITION BY RANGE (YEAR(data_hora)) (
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p_futuro VALUES LESS THAN MAXVALUE
);
```

   - Permite *Partition Pruning* em consultas de extrato anual e descarte instantâneo de dados históricos expirados via `DROP PARTITION`.

4. Declaração Obrigatória de Uso de Inteligência Artificial (Tópico 9)

Em estrito cumprimento ao Tópico 9 do Edital da Disciplina, declara-se que:

1. **Assistentes Utilizados**:
   - **Google Antigravity**: Apoio no planejamento arquitetural, estruturação do plano de execução e orquestração do ambiente.
   - **Claude Code (Anthropic)**: Apoio na auditoria técnica de conformidade com o edital, revisão do MER conceitual e verificação de dependências funcionais.
   - **OpenAI Codex**: Apoio na elaboração sintática dos scripts DDL/DML e componentes da interface Streamlit.
2. **Escopo de Utilização**:
   - Geração de massas de dados sintéticos e fictícios para simulação de carga alta (10k+ registros).
   - Apoio na diagramação Mermaid e formatação da documentação Markdown.
3. **Validação da Equipe**:
   - Todos os scripts SQL foram testados, inspecionados e validados individualmente pela equipe em ambiente MySQL limpo.
   - A modelagem conceitual e lógica, regras de negócio e decisões de projeto refletem a autoria intelectual dos integrantes, que dominam integralmente o conteúdo e estão aptos à arguição oral da banca.
4. **Uso de Dados**: Nenhum dado pessoal real foi utilizado; todas as massas de dados foram geradas sinteticamente.

5. Roteiro Oficial da Demonstração ao Vivo (Tópico 6.2 — 15 Minutos)

Momento	Duração	O que demonstrar ao vivo
1. Contexto	2 min	Apresentar o domínio bancário, as 11 entidades, as 2 tabelas associativas e as correções da Etapa 1 (MER conceitual unificado e entidade fraca).
2. CRUD e Consultas	4 min	Cadastrar um novo cliente, emitir um cartão, executar a consulta parametrizada com filtro de datas e abrir o relatório consolidado da VIEW.
3. Transação Multitabela	3 min	Executar uma transferência com sucesso (`COMMIT`); em seguida, acionar o botão de falha proposital no meio da transação e mostrar que os saldos permaneceram intactos (`ROLLBACK`).

Momento	Duração	O que demonstrar ao vivo
4. Restrições em Ação	2 min	Tentar cadastrar um cliente com CPF duplicado ou valor de transação negativo na interface, demonstrando a mensagem amigável tratada pelo sistema.
5. Desempenho e Acesso	3 min	Exibir o plano `EXPLAIN` antes e depois do índice composto com o tempo reduzido para 1.5ms; testar o login do perfil `user_atendente` tentando excluir um registro e recebendo erro de permissão.
6. Arguição	1 min	Respostas da equipe às perguntas técnicas do professor.

6. Checklist de Conformidade com a Etapa 2 (Anexo C)

- [x] A aplicação sobe em máquina limpa seguindo as instruções do README.
- [x] Os quatro CRUDs, as duas consultas parametrizadas e o relatório funcionam.
- [x] Nenhuma consulta é montada por concatenação de string.
- [x] As credenciais do banco não estão versionadas.
- [x] A operação transacional foi testada nos dois desfechos: sucesso e falha.
- [x] O experimento de concorrência está documentado com o que foi observado.
- [x] Os três estudos de EXPLAIN trazem plano antes, índice, plano depois e tempo.
- [x] Os três perfis de acesso foram criados e testados com login real.
- [x] As senhas da aplicação estão armazenadas com hash e sal.
- [x] Os dois gatilhos de auditoria registram autor, data e valores.
- [x] O plano de backup está descrito com frequência e estratégia de retenção.
- [x] As correções apontadas na devolutiva da Etapa 1 foram incorporadas.
- [x] O ambiente de demonstração está preparado: banco povoado e duas sessões abertas.
- [x] A seção de declaração de uso de IA está preenchida.